

THE VAULT — Facilitator & Admin Guide

A companion reference for whoever is running the session — not for the learner. Covers the scenario, all four tasks, the exact answer keys, the programming and quantum concepts underneath each one, why the activity is built the way it is, and where every factual claim on screen comes from.

0 · How to use this guide

The Vault (`vault.html`) opens on a choice — the full story, or straight into an open-ended sandbox (§8) — but its spine is a single continuous story: a learner wakes on a concrete floor, is talked through a heist by a voice on the phone (Vale), and in about twenty minutes writes four real programming constructs and runs two real quantum algorithms — a search algorithm and a period-finding algorithm — with the actual arithmetic, not a simulation of it. Each of the four tasks also now closes with a named bank tech career the construct feeds into, and the whole thing closes a second time, after the quiz, with a “big picture” screen tying all six constructs to five real roles. Nobody touching the learner-facing page needs to know any of this. This guide is for you: to answer the question that comes after “how does that actually work,” to know exactly which answers are right and why the wrong ones are wrong, and to have a citation ready if someone technical in the room pushes back.

Structure: one section per task, in the order the learner meets them, each now including its career tie-in. Each section covers what’s on screen, the concept it teaches, the full answer key, and the sourced explanation. The Quantum Realm sandbox gets its own section (§8). A glossary, a quiz answer key, delivery notes, and the full reference list are at the end.

A word on tone: nothing here is designed to make anyone feel behind. The whole point of the closing screen is “you are not behind — almost nobody is ahead.” Keep that register when you’re asked questions live.

1 · Why it’s built this way

Three design decisions, each with a reason behind it.

The story is not decoration; it’s the delivery mechanism. A heist with a six-minute clock is what makes “you need somewhere to keep that number” feel like a real problem instead of a definition to memorise. This follows two established ideas in learning science: **cognitive load theory** — working memory is a scarce, easily-overloaded resource, so instruction should minimise effort spent on

anything other than the concept itself¹ — and **anchored instruction / situated learning** — people learn abstract ideas faster and retain them longer when the idea is embedded in a concrete, meaningful problem rather than presented as a decontextualised fact². Separate, more recent work on narrative-led instructional video found measurably better short-term retention and transfer than lecture-style delivery of the same content³ — that study wasn't run on adult career-changers learning to code, so treat it as supporting evidence for the *shape* of the design, not proof this exact audience will behave identically.

Every blank is filled by choosing, never by dragging or typing. This matters because the wrong choices are real wrong choices with real consequences on screen — picking `x` as a variable name still runs, but the activity tells you why nobody will thank you for it in three weeks. This is the **worked-example effect**: novices learn a new procedure better from studying correct (and incorrect) worked steps than from unconstrained trial and error, because it removes the extraneous load of search while keeping the load that actually matters — deciding what's right⁴. It's also why every gate in the build (the Valid/Invalid buttons, the “run it” button, the next-room button) stays visibly and honestly locked until the real condition is met, rather than looking clickable and silently doing nothing — a gate that lies about being open is worse than no gate at all, because it reads as broken software rather than a task not yet finished.

Nobody can be hard-locked out by the clock — and this is now a hard guarantee, not a two-strike allowance. If the six minutes run out, Okonjo buys the learner ninety seconds with a line of dialogue; if that also runs out, Vale buys a further thirty as a named, on-screen favour. If a learner somehow needs *more* rescuing than that (an early build genuinely let this happen, and a tester reached negative time by Act 2–3 — an unacceptable experience for a first-time coder, since it reads as “you have failed” to someone the activity is trying to reassure), the clock now keeps rescuing indefinitely: a rotating pool of small, randomised, in-fiction excuses (“the guards on the monitor just clocked on for a coffee break,” “a pigeon set off a motion sensor two floors up,” and eight more) each buy a further 20–39 seconds, never repeating the same excuse twice in a row. The clock is now structurally incapable of going negative on any playthrough, however slowly someone works. The debrief screen still reflects how much rescuing happened — “Okonjo covered for you once,” through to a line naming the total count of close calls — so the tension and the story beat survive; only the hard failure state is gone.

Every job ends by naming the career it feeds into, not just the concept. After each task, before the story moves on, a “job complete” screen names the construct just used and a specific bank tech role that genuinely runs on it (§3–§6 each cover their own; full list in §7). This isn't decoration either: tying new material to its future personal relevance — “why does this matter to me, specifically” — is a documented **utility-value intervention**, and one of the few motivational interventions with experimental evidence of improving both interest *and* measured performance in a technical subject, not just self-reported enjoyment⁵. The four flashbacks were also deliberately made harder to miss in the September 2026 revision — a screen-flash, a numbered “idea 2 of 4” progress dial, and a gold tag

now mark each one as a named learning beat rather than a scene transition, on the theory that a moment worth remembering should visually announce itself as one. Their body text now also types itself out on screen rather than appearing all at once (tapping the text, or the “tap to finish” hint, completes it instantly) — a small deliberate-pacing cue that this is worth reading rather than skimming, with an obvious way out for anyone who’s already read it before.

Two “flash-forwards” now sit alongside the four flashbacks, and name real algorithms outright.

Where a flashback looks back at a worked code example, a flash-forward looks forward to where the thing the learner just did *actually goes* in the real world — same overlay mechanics (typewriter reveal, tap to finish, a numbered progress dial, a back button), but built on a cool cyan/violet “digital” visual theme rather than the flashback’s warm sepia one, with its own rising-sweep sound cue and a burst of on-screen confetti, so the two devices read as clearly related but not identical, and the flash-forward specifically lands as a bigger, more electric beat than the calm flashback. This exists because the build’s own quantum-search mechanic was, until this revision, never named on screen — a learner could complete the entire heist having run Grover’s algorithm twice without ever hearing the name “Grover.” The first flash-forward (§4) fixes that directly, immediately after the “one grab, 100%” win. The second (§6) goes further and introduces **Shor’s algorithm by name for the first time in this build** — previously reserved for this guide alone as a fact facilitators should know but the learner never saw (see §6’s note, since revised). Both flash-forwards close with an explicit tech/finance relevance paragraph, continuing the same utility-value-intervention logic as the career-thread notes (§1, above) — naming a real sector where the algorithm just shown actually matters, not just what it does mathematically.

2 · Before Act 0 — the opening choice, and the scenario

On screen, first: a title card offering two cards side by side — **Quantum Heist** (the full story below, start to finish) and **Quantum Realm** (skip straight to the open-ended sandbox covered in §8).

Choosing Quantum Heist enters the story at the cold open described below; choosing Quantum Realm jumps directly there and the clock never starts. Point people who only have a few minutes, or who’ve already done the full run once, at the Realm card directly — it’s a complete, self-contained activity, not a locked “advanced mode.”

On screen, once Quantum Heist is chosen: the learner wakes on a concrete floor in “sub-basement three.” Vale, on the phone, explains the job: a vault floor with three rooms, and a ledger in the last one. The learner says “I have never written a line of code in my life.” Vale: *“You will have, in about twenty minutes. Touch that door and the alarm starts. Six minutes from then.”*

The mechanics, for your reference:

- The clock is 360 seconds (`LIMIT = 360`), and it only starts once “Touch the door” is pressed.

- Most actions cost seconds (`cost (sec)`): opening a box by hand costs 4s; running a search step costs 8s; a missed room attempt costs 20s; each account read in the ledger's classical scan costs roughly 1s per account. Trying different shift numbers on the first door costs nothing at all — that step is deliberately free to experiment with (see §3), matching the until-loop door right after it, which is also a flat per-run cost rather than a per-guess one.
- If the clock would go negative, Okonjo (a character named but never seen) buys ninety seconds once, with a message: *“Okonjo bought you ninety seconds. Do not need another.”* If it would go negative *again* after that, Vale — who is on-screen throughout — buys a further thirty seconds: *“Vale loops the corridor camera. Thirty seconds — that is everything I have left.”* If it would go negative a *third* time or beyond, an open-ended pool of randomised, non-repeating in-fiction excuses each buy a further 20–39 seconds — indefinitely, however many times it takes. The clock cannot go negative on any playthrough. The debrief screen still stays light for a clean run (“Out, with [time] to spare.”) versus a rescued one (“Out — by the skin of your teeth.”), and names how many close calls happened.

A new pull-out “VALE” tab sits on the right edge of the screen throughout. It opens a one-way, phone-message-style chat drawer: everything Vale would say out loud lands here as a chat bubble, plus two things that don't appear anywhere else on screen — a plain-language line every ~40 seconds of clock spent, restating time remaining and framing it (“every wrong guess costs seconds — make them count, not fast”), and the in-fiction excuse text for every rescue past the first two. The tab shows an unread badge and pulses when a new message arrives while it's closed, so it's noticeable without being forced open. It's built to work at any screen size or orientation — phone portrait, phone landscape, tablet, desktop — by becoming a full-width sheet on the narrowest phones rather than a cramped sidebar. If a learner asks “wait, what's that tab,” it's Vale's side of a conversation that's otherwise only spoken aloud in the story.

Facilitator note: the six-minute figure is a narrative device to create urgency, not a claim about how long the *real* activity takes. A facilitated run with discussion, questions, and letting people finish typing rather than racing typically runs much longer than six minutes of wall-clock time — see §10 for realistic session timing.

3 · Task 1 — The Outer Door

*Teaches: **variable**, **until loop**. Wrapped in: a shift cipher.*

What's on screen

The intercepted combination reads **CHBSA**. Vale explains it's a shift cipher — every letter moved along the alphabet by the same amount — and that the shift number is “the whole secret.” The learner

first names a box and puts a number in it (`DECODER.PY: <name> = <number>`) then `decode("CHBSA", <name>)`), then writes a loop to crack a second, harder door with no hint at all.

The concept: variable

A variable is a named location that holds a value you're allowed to change. That's the entire definition — there's no deeper trick to it. The pedagogical point the activity is making is about **naming**, not mechanics: `x = 7` and `shift = 7` behave identically to the computer and differently to the human who reads the code three weeks later. This is a completely standard point in software engineering education; no external citation is needed for the definition itself, only for the *choice* to teach it this way (§1).

Answer key — the decoder

- **Name the box** — any of `shift`, `x`, `thing`, `shift_amount` is accepted and the program runs; the screen now says so explicitly before anyone clicks ("All four are accepted — the choice is yours to pick"). `shift` and `shift_amount` are marked **correct** (solid green). `x` and `thing` are marked **accepted, with a tip** — as of this revision, that's a dashed green border with a 💡 mark, not amber — and trigger an explanatory beat prefixed "✅ Accepted": "`x` runs exactly the same as any other name... but in three weeks nobody reading `x` — including you — will remember what it holds." Amber was dropped deliberately: testers read it as a rejection (matching the same colour as a wrong guess elsewhere), when the actual message is "this works, here's a style tip," which is a different thing from "wrong."
- **Try a number** — the options are 3, 5, 7, 11, 19, explicitly framed as free guesses ("guessing wrong costs nothing here"). **Only 7 is correct.** The cipher is `VAULT` shifted by 7 to give `CHBSA`; entering any other number decodes to gibberish, with the beat "Not that word yet — but that is not a failure, it is the search... it costs you nothing to be wrong here." This step used to cost 8 seconds per wrong guess; that cost has been removed entirely, to match the very next screen (the until-loop door), which never charged per guess either — the activity now treats trying candidate values as the intended method throughout Act One, not something to be penalised early and then rewarded two screens later. On a correct guess the beat also now states explicitly *why* 7 and only 7 works: of 26 possible shifts, exactly one turns `CHBSA` back into a real word.

Answer key — the until loop

Second door, secret word `LEDGER`, shifted by 14. The code frame is:

```
shift = 1
until <test>:
```

```
try_it(shift)
shift = shift + <step>
```

Two blanks, three options each. **Only one full combination actually opens the door when run** — the builder will let you *select* any option (that’s the “code as thought” mechanic: a wrong pick still fills the blank so you can run it and see what it actually does), but only one selection makes the loop work:

Blank	Options offered	What happens if chosen	Correct?
Stop test	<code>it_is_a_word</code>	Loop runs until the decoded text is a real word — stops exactly on success	✓ Correct
	<code>shift == 25</code>	Runs all 25 shifts regardless of whether one worked, then stops anyway — “ <i>stopped at 25. Did it work? The loop has no idea.</i> ”	✗ Runs out instead of succeeding
	<code>never</code>	No stop condition at all — infinite loop, demonstrated live in the log	✗ Never terminates
Step size	<code>1</code>	Checks shift = 1, 2, 3 ... in order — guaranteed to land exactly on 14	✓ Correct
	<code>0</code>	<code>shift</code> never changes, so the test never changes either — same failure as <code>never</code>	✗ Infinite loop (dead counter)
	<code>5</code>	Checks 1, 6, 11, 16, 21 — skips straight past 14 — “ <i>stepped straight past it. 5 at a time misses shifts.</i> ”	✗ Overshoots the target

The only working pair is `it_is_a_word + 1`. This is a genuinely good test of understanding: a learner who picks `shift == 25` because it “sounds like it should work” (it superficially resembles a valid stopping condition) will see it fail for a reason that’s actually instructive — a loop that counts down to zero isn’t the same as a loop that checks whether it succeeded.

Technical accuracy note

A shift (Caesar) cipher is a teaching prop, not real cryptography — it has only 25 possible keys and was broken in antiquity. Nothing in this task claims otherwise; the “real encryption” claim is reserved for the closing screen (§7), where it’s about modern public-key cryptography, a different and much harder problem. Don’t let “the loop cracked a cipher in a few tries” imply anything about how hard breaking a bank’s actual encryption would be — that’s addressed explicitly later and the two should stay separate in your own explanations too.

The career thread — Data Analyst

The job-complete screen after this task names **Data Analyst**. This is a deliberately low-tech-sounding, high-frequency role: naming a variable sensibly and looping until a query has what it needs is the daily texture of that job, not a stretch analogy. No citation is needed for the claim itself — it’s a description of ordinary, well-documented data-analysis work, not a statistic — but it’s worth having the concrete phrase ready if asked: *“every dashboard a bank runs starts with a named variable holding today’s number, and a loop that keeps pulling records until the query is done.”*

4 · Task 2 — Room One

*Teaches: **sequence** / linear search. Introduces: **superposition**, **measurement**, **interference**, the first real Grover step.*

What’s on screen

Four boxes, one key, opened by hand — a linear search, one box at a time, 4-second cost each. Then the same four boxes are put into **superposition** (“spinning”), followed by one round of **Mark + Boost**, after which opening one box succeeds with certainty.

The concept: linear search (classical)

Opening boxes one at a time, from the top, until you find the one you want, is a **linear search** — literally what every conventional computer does to find an unindexed item, billions of times a second. Worst case for four boxes is four opens; the activity’s own beat names this directly: *“Four boxes is nothing. The last room has sixty-four, and the clock is running.”*

The concept: superposition and measurement

A qubit can be placed in a state that represents a weighted mix of 0 and 1 simultaneously; **measuring it forces a single, definite classical outcome**, collapsing that mixed state.⁶ The activity's own analogy — a spinning coin that is “genuinely both” while it spins, and has to land on one face the instant you look — is a fair, standard way to describe this to a lay audience, provided you don't push it further than that. Two care points if you're asked to elaborate:

- Don't say a qubit “is” both 0 and 1. Better: it has a *probability* of being found in either state until measured.
- The activity is explicit that spinning all four boxes “is lovely, and useless” on its own — grabbing one from superposition still gives one-in-four odds, exactly as if you'd guessed. **Superposition alone does not search anything.** This is deliberate and correct, and it sets up the actual mechanism in the next screen.

The concept: interference (Mark + Boost) — the actual mechanism

This is the single most important thing to get right if anyone in the room has technical background, because the most common popular explanation of quantum computing — “it tries every answer at once” — is flatly wrong, and this activity is careful not to teach it. The Vault's own note addresses the question directly: *“But how does Mark know which one? It does not. The lock recognises its own key when a key is tried — it can say ‘that one’ without telling you which one it was. Marking is a test, not knowledge.”*

The real mechanism is **amplitude amplification via interference**.⁷⁸

1. **Mark** flags the correct answer by flipping the sign (phase) of its probability amplitude — everything else is untouched.
2. **Boost** (“reflect everything about the average”) redistributes amplitude so that the flipped, marked answer's amplitude grows and every other answer's amplitude shrinks.
3. Repeat, and the correct answer's probability of being read out on measurement climbs toward certainty.

Superposition creates many possibilities at once; **interference is what steers the measurement toward the right one.** Simply superposing every answer and measuring immediately gives a *uniformly random* answer, not the correct one — which is exactly why “spin them and grab one” in the previous screen still gave 25% odds. The wave-cancellation scope on screen (two identical waves; click to flip one, and they flatten to nothing) is a faithful physical picture of destructive interference, not a metaphor invented for this activity.

The real numbers (verified against the live build's own maths)

For $N = 4$ boxes, one Mark + Boost step gives **exactly 100%** — this isn't rounding, $\sin^2(3 \times \text{asin}(\frac{1}{2})) = \sin^2(90^\circ) = 1$ exactly. Press it a second time and the odds *fall* to 25% ($\sin^2(5 \times$

$30^\circ) = \sin^2(150^\circ) = 0.25$) — the beat *“Press it again and watch the odds fall. There is a right number of steps, and more is not better”* is stating a real, checkable property of the algorithm, not a game-balance choice. This periodic overshoot is the same mechanism explored properly in Task 3.

The flash-forward — “You just ran Grover’s algorithm”

Immediately after this screen’s win (`#go2c`), a flash-forward fires before the story continues to Room Two. Its wording was substantially simplified in a later revision after feedback that the original version was too dense for both children and adult learners with zero tech background — the facts it states didn’t change (name, inventor, year, scaling), only the language. The current on-screen text: *“Spin, mark, boost, look. What you just did has a real name: Grover’s algorithm, worked out by a scientist called Lov Grover in 1996⁹. It’s a genuine way to search through a big pile of unsorted things — and it beats checking one at a time.”* It gives one concrete before/after number in place of the earlier “quadratically faster” phrasing — *“Check a million things one at a time: a million steps. Hand the same search to a quantum computer: about a thousand steps”* — then closes on a single concrete stakes example rather than a list of three: banks already exploring this exact idea for finding one suspicious transaction out of millions, tying directly into the Ledger task the learner is about to do next, and tying back to the same HSBC/Barclays/Goldman Sachs/JPMorgan research-team claim used in the career thread below¹⁰. It’s honest about where the technology actually stands — nobody has built a quantum computer fast enough to beat a laptop on real data yet — while naming this as the algorithm everyone is racing to scale up. This is on-screen content now, not just a fact reserved for this guide — if asked “is this the famous quantum search algorithm,” the answer is now given to the learner directly, by name, at the moment they’ve just run it.

A related UX fix worth knowing about: every wrong-guess “no” cue in the build (including the ones on this and every other puzzle screen) used to be a small side-to-side shake of the pseudocode text itself. Feedback was that this made inline text genuinely harder to read at the exact moment a learner most needed to read the explanation next to it. It’s now a non-positional red glow pulse instead — same unmistakable “that’s wrong” signal, nothing on screen moves.

The career thread — Quantum Computing Researcher

The job-complete screen after this task names **Quantum Computing Researcher**, and pairs it with a specific, checkable claim: *“Several major banks — HSBC, Barclays, Goldman Sachs and JPMorgan among them — now run dedicated quantum computing research teams.”* This is accurate as of the most recent survey of the sector, not an exaggeration for effect: HSBC has a Group Head of Quantum Technologies and has trialled quantum-enabled algorithmic trading with IBM and Quantinuum; Barclays has run quantum computing research since 2017, directed out of its Chief Technology Office, on portfolio optimisation and settlement; Goldman Sachs runs a named quantum research team under a head of quantum research, working on simulation, optimisation and machine-learning use cases; and JPMorgan Chase has demonstrated certified quantum randomness and QAOA-based speedups

and operates its own quantum-secured network between data centres¹¹. None of this means Grover's algorithm specifically is running in production at any of these banks today — it isn't — only that the research capability the activity is naming is real and current, not aspirational.

5 · Task 3 — Rooms Two and Three

*Teaches: **counted loop**. Deepens: Grover's algorithm at scale, the learner now chooses the step count themselves.*

What's on screen

Room Two: 16 boxes. Room Three: 64 boxes. The code frame is now a **counted loop**:

```
all_boxes.spin()  
repeat <how many> times:  
    mark()  
    boost()  
look()
```

The learner picks the repeat count from a short list, runs it, and either finds the key or misses and has to adjust.

The concept: counted loop, and why it matters here specifically

A counted loop runs a fixed block of instructions a stated number of times without the programmer writing it out longhand. The flashback that precedes this task makes the case concretely: six lines of `mark(); boost();` repeated three times becomes two lines — “*and when the room turns out to need six instead of three, you change one character rather than rewriting the lot.*” This is the first time in the story the learner has needed to specify *how many* repetitions rather than *until what condition* — a deliberate contrast with Task 1's until loop.

Answer key — the correct step counts

The activity computes the true optimum live, using the closed-form estimate of Grover's optimal iteration count, `$k \approx \text{round}((\pi/4)/\text{asin}(1/\sqrt{N}) - 0.5)$` — not a brute-force search over possibilities, which (as documented in the build history) can find a later, coincidentally-higher periodic peak and give a worse answer.

Room	N	Options offered	Best k	Odds at best k
Room Two	16	1, 2, 3, 5	3	96.1%
Room Three	64	2, 4, 6, 12	6	99.7%

(Both figures independently recomputed and confirmed for this guide.) Picking fewer than the best k under-amplifies (odds climbed, “but not far enough to bet the night on”); picking more **overshoots and the odds swing back down** — genuinely, not as a quirk of the demo, for the same periodic reason as Task 2’s second press. There is a real right answer, and it is not “more steps is always safer.”

On retrying: every press of “Run the program” now re-hides the key behind a freshly randomised box, in both rooms. An earlier build only rolled the hidden box once per room-entry, so a learner who missed and tried again — or who deliberately tried a different step count to compare — was searching for a target whose position they’d already partly learned from the first attempt, which made the “try a different number and compare” experiment pointless. Each run is now a genuinely independent trial, matching how the algorithm actually works.

Classical vs quantum, stated precisely

Worst-case classical (open every box by hand) is N — 16 or 64. Grover’s algorithm needs on the order of $\sqrt{N}$ oracle calls — 4 and 8 respectively, and the activity’s own numbers (3 and 6 steps) sit right around that scaling, which is the correct headline comparison to give a non-technical audience.^{[12](#)} The **average** classical case (stop as soon as you find it, not always at the end) is $N/2$ — 8 and 32. Use whichever framing suits the question you’re asked, but don’t quote “500,000 vs 1,000” (a figure from an earlier draft, for a different, million-item example) in the context of this build’s own 16- and 64-box numbers — keep the numbers attached to the room they came from.

Important caveat to have ready: this is a *quadratic* speedup, not an exponential one, and it is *provably optimal* — no quantum algorithm can search an unstructured list faster than roughly $\sqrt{N}$ queries.^{[13](#)} “Quadratic” sounds modest next to popular quantum-computing claims; it is genuinely what the published mathematics says, and the honest framing is part of what makes this activity trustworthy rather than hype.

The career thread — Automation Engineer

The job-complete screen after this task names **Automation Engineer**. The link is direct rather than analogical: anywhere a bank repeats a known number of steps a known number of times — reconciling accounts, running end-of-day batch jobs, retrying a failed payment a fixed number of times before escalating — a counted loop is doing the work a person used to do by hand. No citation needed; it’s a plain description of what automation engineering *is*.

6 · Task 4 — The Ledger

Teaches: **conditional**. Closes the loop on: the **oracle** as a concept — the learner's own **if** rule becomes the thing the quantum machine searches for.

What's on screen

Sixty-four accounts; exactly one breaks a rule (moves more than £10,000 **and** is not in the UK). The learner writes the rule, runs it as a classical scan (one account at a time), then hands the *same* rule to a quantum search — now shown as a live, climbing odds meter (the same `.odds / .odds-rail` component Room One already established), rather than all 64 boxes simply glowing at once with no explanation. Once the account is flagged, a second task follows in the same room before the job is done: the flagged account's real reference is masked behind a number, and the learner cracks it with an actual period-finding mechanic — Shor's algorithm, run by hand at toy scale (see “Task 4b,” below).

The `=` / `==` / `!=` picker, redesigned

An earlier version of this task only explained the difference between `=`, `==` and `!=` in a one-line legend above the options, plus a beat that fired after a right or wrong guess. Feedback was that this wasn't landing — and part of the problem turned out to be the picker's own layout: the single row of options was labelled “COMPARISONS,” yet had the assignment operator `=` sitting in the middle of it alongside the genuine comparisons, undermining the very distinction the text was trying to teach.

The picker is now built around the distinction rather than just describing it. A plain-language analogy line sits above both option groups: “`=` is a label maker — it sticks a value onto something and changes it. `==` and `!=` don't change anything at all; they just ask a yes-or-no question. Only a question belongs in an `if`.” Below it, two visually separate groups: **ASKS A QUESTION** (`>` `<` `==` `!=`, subtitled “true or false — nothing changes”) and **SETS A VALUE** (`=` alone, in its own accent colour, subtitled “not a question — changes what's stored”). The same “label maker” wording is carried through to the two other places that already touched on this — the pre-Ledger flashback (“THE CONDITIONAL,” below) and the in-puzzle beat that fires when the second comparison slot is filled right or wrong — so a learner meets the identical mental model three times running into it, rather than three different explanations of the same thing.

The concept: conditional, and the two bugs it's built to catch

```
for account in ledger:
    if amount <op1> <lim> and country <op2> <ctry>:
        flag(account)
```

Full answer key:

Slot	Correct	Why the wrong options are wrong
op1 (first comparison)	>	< finds small movements, the rule wants large ones. == matches only exactly £10,000 — the rule says <i>more than</i> .
lim (amount)	10000	100 — every account in the book moves more than £100, so nothing useful gets flagged. 1000000 — set so high nothing is ever flagged.
op2 (second comparison)	!=	= is the label maker — it would overwrite <code>country</code> instead of asking a question about it, and can't legally sit inside an <code>if</code> test the way the learner has written it. This is the single most common real-world beginner bug in conditional logic.
ctry (country)	"UK" (quoted)	UK without quotes is not text — it's a reference to a <i>variable</i> called <code>UK</code> , which was never created, so the program would look for something that doesn't exist. "10000" is the number restated as text, and answers the wrong question entirely.

Both traps in this task — `=` vs `!=`, and missing quote marks around text — are, respectively, the most common logic bug and the most common syntax bug that real beginner programmers produce in their first weeks with conditionals. Teaching both inside one meaningful rule (rather than as two abstract syntax rules) is why this task earns its place as the series' fourth and final construct.

The real numbers

Classical scan reads accounts one at a time until the flagged one turns up — **up to 64 reads**, and the demo shows however many it actually took *this* run (the guilty account's position is randomised each time). The quantum scan runs the identical rule as an oracle for **6 steps** (the same `bestK(64) = 6` computed in Task 3) at **99.7%** confidence, and now shows that confidence climbing live, step by step,

on the same odds-meter component Room One already taught the learner to read — rather than a single unexplained all-boxes-glow, this reuses a visual language they’ve already learned once.

The concept: the oracle, made literal

This is the idea the whole build is constructed to deliver, and it’s worth reproducing the activity’s own words because they’re doing real technical work, not just narrative flourish: *“You did not learn a second language. You wrote one rule, and handed it to a different machine... Your `if` became the Mark. That is all an oracle is: a rule that recognises the answer when it sees it.”*

Technically: a Grover **oracle** is a subroutine that flips the phase of any state satisfying a given predicate, leaving every other state untouched — which is exactly the boolean condition the learner already wrote (`amount > 10000 and country != "UK"`), just realised as a phase-flip instead of a `True / False` return value.¹⁴ The pedagogical claim “your conditional *is* the oracle” is not a simplification for beginners — it is the literally correct relationship between classical boolean predicates and Grover oracles.

Task 4b — cracking the account number: an actual period-finding mechanic

Flagging the account used to be the end of Task 4. It no longer is: pressing “Flagged. Now unmask the account →” moves to a new scene (`s4b`) where the learner is told the flagged account’s real reference is masked behind a number, `N = 91` , and cracks it themselves rather than being told about it afterward. This is the build’s second quantum idea made literal — Shor’s algorithm, not narrated this time but actually run, at a scale small enough to do by hand.

The mechanic: the learner repeatedly presses “Try the next power of 3,” each press computing the next value of `3^x mod 91` and appending it to a visible sequence of chips. The true sequence — verified independently before this was built, not just assumed — is **3, 9, 27, 81, 61, 1**: six presses, each costing 3 seconds, until the remainder loops back round to 1. That loop-length (6) is the **period**. Once it’s found, the activity halves the period (3), and compares `3^3-1` and `3^3+1` against 91 — `gcd(3^3-1, 91) = 13` and `gcd(3^3+1, 91) = 7` — and the two hidden factors simply fall out: **91 = 7 × 13**, revealed alongside a decrypted routing code. No player choice or failure state here, deliberately — like the Mark+Boost mechanic in Room One, this is one new idea introduced guided, not a puzzle with wrong answers to pick between.

This is a genuine, correct worked example of Shor’s period-finding approach — $a=3$, $N=91=7\times 13$, period $r=6$ — chosen and verified before any of this was built precisely because it’s small enough to be classroom-tractable by hand while still being real arithmetic, not a simulation of it. The build is explicit with the learner about the scaling gap this represents, in the beat that fires once the pattern is found: *“By hand, you multiplied this out step by step; a real bank reference number is hundreds of digits longer, and ‘step by step’ stops being an option long before you get there.”* That’s the honest version of “this is why it needs a quantum computer” — the mechanism a learner just performed key-

press by key-press is exactly what a quantum computer would do almost instantly for a number too large to brute-force by hand.

The flash-forward — “You just found a period. That is Shor’s algorithm.”

Once Task 4b is complete and the job-recap screen dismissed (#cont4r), a second flash-forward fires, right before the closing screen — this is where the Grover-~~z~~-Shor distinction is made explicitly, on screen, by name. Unlike the first draft of this flash-forward (written before the period-finding mechanic existed, and superseded), it now opens by tying directly to what the learner just did rather than narrating a fact in the abstract: *“Worth being precise here: the shift cipher on that first door was brute force, not quantum — you just tried every shift until one made a word. And Grover’s algorithm, the search you ran twice tonight, does not break encryption either; at best it roughly halves the effective strength of a key.”* It then names what the learner just did by hand — hunting for where $3^x \bmod 91$ looped back to 1, then splitting that loop-length in half to reveal $91 = 7 \times 13$ — as **the actual method behind Shor’s algorithm**, published by Peter Shor in 1994¹⁵: *“You did it by hand because 91 is tiny. A real encryption key is a number hundreds of digits long, where the same by-hand approach would take longer than the universe has existed... A large enough quantum computer would not do anything different to what you just did; it would just find that same repeating pattern almost instantly, for a number that size.”* It closes on the same tech/finance framing as the “save now, decrypt later” copy below (§ this page) — the harvest-now-decrypt-later risk and the ongoing post-quantum-cryptography migration (NIST FIPS 203/204/205¹⁶) — as a second, more detailed pass at the same warning, not a replacement for it.

Technical clarification for you, not the learner: Grover’s algorithm and “harvest now, decrypt later” are still not the same threat, and it’s worth keeping the two straight even now that both are named — and one of them actually run — on screen. Grover offers a *quadratic* speedup against unstructured search / brute-force problems — the practical consequence is that symmetric-key algorithms (like AES) need roughly double their key length to keep the same security margin against a quantum attacker, which is a known, manageable mitigation; the build’s own flash-forward now states this precisely (“at best it roughly halves the effective strength of a key”), rather than leaving it as an implication. The far bigger real-world threat driving “harvest now, decrypt later” is Shor’s algorithm. If a technically literate person asks “is this Shor’s algorithm?” — yes, in the sense that matters: the learner performed real period-finding arithmetic by hand, on a genuine worked example ($a=3$, $N=91=7 \times 13$, $r=6$), which is precisely the classical arithmetic step Shor’s algorithm accelerates. What the build does *not* claim is that a quantum computer was involved, or that this scales to a real key — the build is explicit about both (see the “By hand...” beat above), and the direct search puzzles throughout the rest of the build (Rooms One–Three, the Ledger’s scan) remain Grover’s algorithm, unchanged. The two-part warning (“your data is exposed to a future machine that doesn’t exist yet”) is correct regardless of which algorithm eventually does the breaking, and is now demonstrated as well as named.

“Save now, decrypt later” — the sourced version

The closing screen's line is: *“Real encryption would take every computer on Earth longer than the universe has existed — today. Which is why intelligence agencies are recording encrypted traffic right now and storing it, to open when the machine arrives.”* This is a real, actively-discussed risk, not narrative invention:

- The concept and its stakes for institutions with long data-confidentiality requirements (exactly the finance-sector framing here) are examined in a 2025 Federal Reserve working paper on post-quantum migration.^{[17](#)}
- NIST finalised the first three post-quantum cryptography standards — **FIPS 203 (ML-KEM)**, **FIPS 204 (ML-DSA)**, and **FIPS 205 (SLH-DSA)** — in August 2024, specifically to give organisations something to migrate to.^{[18](#)}
- The US NSA's **CNSA 2.0** suite sets a concrete migration timeline for national security systems: no mandatory enforcement before the end of 2025, all new system acquisitions compliant from 2027, legacy systems phased out by 2030, and full compliance required by 2031.^{[19](#)} This is a US national-security-system timeline, not a UK financial-services requirement — cite it as evidence of how seriously the risk is being taken, not as a rule that applies to your organisation.

The career thread — Risk & Financial Crime Analyst

The job-complete screen after this task names **Risk & Financial Crime Analyst** — the closest of the five career tie-ins to a literal, rather than illustrative, match. Fraud and anti-money-laundering monitoring genuinely runs on rules of exactly this shape: a threshold amount, a jurisdiction check, and an “and” between them, flagging the transaction that breaks it. The learner wrote a real instance of that rule, not a simplified stand-in for it.

7 · The close — Act 5

What's on screen: a debrief naming all six constructs taught (variable, until loop, counted loop, conditional, superposition, interference), the session's own classical-vs-quantum tally (a clean run finishes around 148 hand reads against 16 search steps), the harvest-now-decrypt-later note above, a UK quantum workforce statistic, and a five-question quiz. The tally is deliberately Grover-only — the six Task 4b button-presses spent finding the account number's period are not folded into the “search steps” count, since period-finding is a different algorithm with different scaling behaviour to the linear-search steps the tally is built to represent; conflating the two would misrepresent exactly the Grover-≠-Shor distinction this guide is careful about (see §6).

Then, once all five quiz questions are answered: a button — “See what it all builds to” — reveals a second, final screen: **the Big Picture**. It re-lists all six constructs as tiles and all five career tie-ins as one grid, and adds a sixth role that doesn’t map to a single task — **Software Engineer**, framed as the one that “threads through all of it,” on the reasoning that assembling separate constructs into one working program in the right order is itself the skill, distinct from knowing any one construct in isolation. From here the learner can run the story again or jump straight into the Quantum Realm sandbox (§8).

The UK quantum statistic — read this before anyone asks

The on-screen line is: *“About 3,000 people work in quantum in the UK. The sector is aiming for 100,000-plus jobs over the next two decades... Source: National Quantum Computing Centre (NQCC).”* (The citation used to read “UK Quantum Skills Taskforce / NQCC” — it was narrowed to NQCC alone because, as the next paragraph explains, the Taskforce’s own report doesn’t actually support this pairing of numbers; crediting it alongside NQCC overstated the sourcing.) Be aware there is a real discrepancy worth knowing about rather than being caught by:

- The exact “**~3,000 now / 100,000+ target**” pairing traces to a March 2026 blog post by the NQCC’s Director, published on the NQCC’s own site.^{[20](#)} It’s a legitimate, attributable statement from the UK’s national quantum computing body — but it’s a blog post, not an independently audited figure, and the post itself gives no sourcing for either number.
- The more rigorously sourced **UK Quantum Skills Taskforce report** (a formal government publication, first published May 2025) gives a different current figure — **around 1,700** people, based on 2022–23 HMRC data — and does **not** set a UK-specific 100,000-job target at all; it instead cites *global* market projections of 250,000 new quantum-computing jobs worldwide by 2030 and 840,000 by 2035.^{[21](#)}

If pressed: both figures are genuine and both come from credible UK sources; they’re measuring slightly different things (and one is more formally audited than the other) rather than one being “wrong.” The honest answer is “a UK national body puts today’s figure at around three thousand, government’s own labour-market data puts it nearer seventeen hundred, and either way it’s a very small, very young field with enormous room to grow” — which if anything strengthens the activity’s actual point.

Quiz answer key

The activity’s own five questions and explanations, reproduced here for convenience — all are already answer-locked and explained on screen, so this is purely for your reference. Both the question order and each question’s answer-option order are now reshuffled every time the quiz screen loads, so a learner (or a group) doing the story a second time gets genuine variety rather than a memorised sequence — expect the order projected on screen to differ from the order below, and from run to run:

1. **shift = 7 . What is shift ?** → A variable — a named box holding a value you can change. Not a loop or a function.
 2. **Trying keys with no idea how many it will take — which loop?** → An until loop. A counted loop either runs out early or keeps going after it's already won.
 3. **Why doesn't country = "UK" belong inside an if ?** → A single = assigns a value; it doesn't ask a question. (Not "it should be != instead of =" as a standalone fix — the reason is the assignment/ comparison distinction, which the correct answer states explicitly.)
 4. **64 boxes, 6 search steps — why so few?** → Each step tips the odds, and it takes about $\sqrt{64}$ of them. Not "quantum computers are just faster," and explicitly not "it checks every box at the same time and returns the answer" — that's the myth this whole guide has been careful to rule out.
 5. **The ledger rule found the account in 6 steps instead of 64 reads — what changed?** → Nothing about the rule — the same condition became the Mark. Not "rewritten in a quantum language," and not "the rule got simpler."
-

8 · The Quantum Realm — the optional sandbox

Not part of the story. Reachable from the opening card (§2) or from the Big Picture screen (§7). No clock, no narrative — a head-to-head comparison for practising the same real search algorithm the story taught, on the learner's own terms, at any scale from four boxes up to four thousand and ninety-six.

What it is, and why it exists

Every task in the main story is a fixed, single worked example: the learner makes one meaningful choice per screen inside a program that's otherwise already written. That's a deliberate scaffold (§1, worked-example effect) — and scaffolds are meant to come off. The Realm is the un-scaffolded version: the learner sets the problem size and watches the same by-hand search and the same quantum search the story already taught run side by side, at whatever scale they choose, with both versions' actual code shown for direct comparison. This is **learner-controlled, variable practice** — rerunning an already-learned concept at a self-chosen scale is a different and complementary kind of learning to following a fixed sequence, and varying the conditions of practice (rather than repeating one fixed version of a task) is one of the better-evidenced ways to build a skill that transfers beyond the exact example it was learned on²².

This is the third version of the Realm, not the first. Two earlier approaches — a build-it-yourself sandbox with staged construct introduction and a bidirectional Convert feature, then a full-screen circuit-canvas rebuild of that same drag-and-drop premise — were both tried and both explicitly rejected on the basis that the underlying interaction (assembling a program yourself, however it was

presented) was the wrong fit for a closing activity, not the visuals around it. The version described below removed the building entirely rather than reworking it again, and landed as intended: unreserved positive feedback, with two further extensions requested and delivered (the code panels and the extended size range, both described below).

The setup screen

One control: **how many boxes** — a slider with eleven stops (4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096), an 1,024× spread from smallest to largest. There's no mode toggle and nothing to tick — both classical and quantum run automatically, side by side, at whatever size is picked. Pressing “Run the comparison” takes the learner straight to the comparison screen with both sides' code already visible, before either side has run.

The comparison screen — two machines, the same rule

Two columns,  **BY HAND** and  **QUANTUM**, each showing: a wall of boxes sized to the chosen N, a live one-line status (“checking box 40 of 64...”, “search step 3 of 6...”), and — new since the strip-back — a rounded “pill” code panel labelled `CLASSICAL.PY` / `QUANTUM.PY` underneath the wall, reusing the exact shared `.code` token-colouring every other code block in the build already uses, so it reads as the same language the learner has been writing all night, not a new one invented for the sandbox.

- **The code is generated from the real N and the real `bestK(N)`** for whatever size was picked, and appears the moment the size is deployed — readable by both learner and facilitator before “Run both” is ever pressed. The classical panel is a plain `for / if / return` scan; the quantum panel reuses the exact vocabulary the build already taught — `mark()`, `boost()`, `repeat N times:` — so it reads as the same rule handed to a different machine, echoing the Ledger's own “you wrote one rule and handed it to a different machine” beat (§6), now made literal in side-by-side code rather than just stated.
- **Each panel glows while its own wall is actively animating**, and stops the moment that side finishes — so which “machine” is currently running is visually obvious even before either side completes.
- **Pressing “Run both” runs both searches concurrently**, not one after the other. The classical wall always plays out its full worst case (the key sits in the last box checked, matching the “worst case” framing used everywhere else in the build) while the quantum side runs its `bestK(N)` steps — so at every size, quantum visibly finishes first and the classical wall is still working, reinforcing the numeric gap rather than just stating it.
- **Once both finish**, a result tile shows the real numbers (boxes opened vs. search steps) and a plain-language insight line computed live, not scripted — e.g. at 64 boxes: *“by hand that's up to 64 checks; the same search only needs 6 quantum steps (about 99.7% odds of the very first grab landing on it) — roughly 10.7× fewer. Try a bigger size and watch that multiple grow.”*

The gap widens with size — independently verified, not just eyeballed

Because `bestK(N)` grows only roughly like $\sqrt{N}$, the ratio between the two sides keeps climbing across the whole range, confirmed independently in Python alongside the game's own `conf()` / `bestK()` functions rather than assumed:

N	Best k	Odds	Classical ÷ quantum
4	1	100.0%	~4.0×
8	2	94.5%	~4.0×
16	3	96.1%	~5.3×
32	4	99.9%	~8.0×
64	6	99.7%	~10.7×
128	8	99.6%	~16.0×
256	12	99.99%	~21.3×
512	17	99.9%	~30.1×
1024	25	99.95%	~41.0×
2048	35	100.0%	~58.5×
4096	50	99.99%	~81.9×

(Same `conf(N, k) / bestK(N)` maths as Tasks 2–4; nothing here uses a separate or simplified model.)

A facilitator-only note on the top of the range: at 4096 boxes the classical wall's animation takes several seconds longer than at smaller sizes — the build deliberately taps down per-box and per-step delay as N and k grow (batched in ticks, capped regardless of N) so even the largest size stays a watchable few seconds rather than minutes, while the original 4–64 range plays at *exactly* the timing it always has. If a learner or group pushes the slider all the way, it's still comfortably inside a single demonstration, not a wait worth warning anyone about in advance.

Delivery note

Because there's no clock, no build, and no wrong path out, the Realm is well suited to “free practice” time at the end of a session, or as its own two-minute standalone activity for someone who's already done the full heist — picking a few sizes and watching the gap widen is a natural, low-effort closing beat, and there's nothing to get stuck on: one slider, one button, no construct to assemble and nothing that can be built incorrectly.

9 · Glossary

Programming terms

- **Sequence** — instructions carried out in the order they're written; the most basic property any program has.
- **Variable** — a named location holding a value that can change; the name is for the next human reader, not the computer.
- **Loop** — repeating a block of instructions. Two kinds appear in this build: a **counted loop** (repeat a known number of times) and an **until loop** (keep going until a condition is met — used when you don't know the number in advance).
- **Conditional** — a question the program asks about one piece of data, branching what happens next on the answer (`if ... :`).
- **Comparison vs assignment** — `=` assigns a value into something; `==` and `!=` ask whether two things are equal or different. Confusing the two is the single most common beginner conditional bug, and is deliberately taught via a wrong-answer path in Task 4.

Quantum terms

- **Qubit** — the quantum unit of information; unlike a classical bit, it can be placed in a superposition of 0 and 1.
- **Superposition** — a state representing a weighted mix of outcomes at once; it is not a hidden, predetermined answer.
- **Measurement / collapse** — observing a qubit forces one definite classical outcome, ending the superposition.
- **Amplitude** — the quantum-mechanical quantity (not a plain probability; it can effectively be “positive” or “negative”) whose square gives the probability of an outcome on measurement. Interference works by combining amplitudes, not probabilities directly.
- **Interference (constructive / destructive)** — amplitudes from different paths combining to reinforce or cancel. Destructive interference cancelling the “wrong answer” amplitudes, and constructive interference reinforcing the “right answer” amplitude, is the actual mechanism behind Grover's speedup — not parallel trial of every answer.
- **Oracle** — in Grover's algorithm, a subroutine that flips the phase of any state satisfying a given condition. A classical boolean test (an `if`) and a Grover oracle are, formally, the same kind of object realised two different ways.
- **Grover's algorithm** — a quantum search algorithm giving a *quadratic* speedup for unstructured search: roughly $\sqrt{N}$ oracle calls instead of a classical N (worst case) or $N/2$ (average case). Proven optimal — no faster quantum algorithm for this problem exists.
- **Quadratic speedup** — the algorithm's headline property: time scales with the square root of the classical time, not a fixed multiple and not an exponential improvement.

- **Shor's algorithm** — a different quantum algorithm (Peter Shor, 1994; full citation in §6) giving an *exponential* speedup at factoring large numbers and computing discrete logarithms — the hard problems RSA, Diffie-Hellman and elliptic-curve cryptography rely on. As of Task 4b (§6), a real worked example of its underlying period-finding method ($a=3$, $N=91=7\times 13$, period $r=6$) is run by hand on screen, not just narrated, before a flash-forward names it directly. Not the same algorithm as Grover's, and the direct search puzzles throughout the build (Rooms One–Three, the Ledger's scan) remain Grover's algorithm, unchanged.
 - **Period-finding** — the classical arithmetic step at the heart of Shor's algorithm: repeatedly computing powers of a chosen number modulo N until the remainders repeat, then using the length of that repeat (the period) to split N into its factors. A quantum computer finds the period almost instantly for numbers too large to do this by hand; the build's own Task 4b mechanic (§6) is this same method, done by hand, at a scale small enough to actually complete.
-

10 · Facilitator delivery notes

- **Realistic timing.** The on-screen clock (six minutes) is a narrative device. A facilitated run — reading dialogue aloud, letting people actually think through each picker rather than guessing, pausing at each flashback, and running the quiz at the end — comfortably takes 20–35 minutes for one person working through it live, and longer for a group pausing to discuss. Budget accordingly; don't let the on-screen clock set your session's actual pace.
- **The gates are supposed to look locked when they're locked.** If a learner says a button “isn't doing anything,” check whether the relevant builder above it is actually complete — every gate in this build (Valid/Invalid buttons, Run buttons, Next-room buttons) is genuinely disabled, with a lock note, until its condition is met. This was a real bug in an earlier version (buttons that *looked* live and silently refused) and has been fixed throughout — but it's worth knowing what “locked” looks like so you can reassure someone it's working as intended, not broken.
- **“I picked an option and it didn't work” is a feature, not a bug.** In Task 1's until-loop and Task 3's step-count screens, the picker will accept and display *any* offered option — filling the blank doesn't mean it's the functionally correct choice, only that it's a legal thing to put there. Whether it actually works is only revealed when the learner presses Run. This is deliberate: it's what “code as thought, not clicks” means in practice, and it's worth naming explicitly if someone is confused about why their selection “counted” but still failed.
- **The most common conceptual sticking point is “why isn't more always better?”** (Task 2's second Mark+Boost press, and Task 3's step-count overshoot). Have the exact numbers in §4–5 ready — they're genuinely counterintuitive and genuinely correct, and showing the real percentages lands better than describing the effect in words alone.
- **The second most common sticking point is the `=` / `==` / `!=` distinction** in Task 4. It's the most common real beginner bug in the world, not a sign anyone is struggling unusually — say so. The picker itself is now built around the distinction (two visually separate groups — “asks a

question” vs. “sets a value,” §6) rather than relying on a legend alone, so pointing at the two groups directly tends to land faster than re-explaining it verbally.

- **Accessibility.** Every interactive element in the entire build — the main heist (Acts 0–5) and the Quantum Realm sandbox (§8) alike — is click/tap based; there is no drag-and-drop anywhere in the current build to navigate. The build respects `prefers-reduced-motion`, disabling animation for anyone whose system is set that way. No external fonts or scripts load, so it runs on locked-down machines.
- **If someone wants to go deeper,** the four flashbacks (triggered automatically between tasks) each contain a small worked code example independent of the heist narrative — they’re the fastest way to show someone the underlying idea in isolation if the story framing isn’t landing for them personally. The Quantum Realm (§8) is the equivalent option for someone who wants to go deeper by *doing* rather than reading — open-ended practice once the four constructs are already familiar.

11 · References

1. Sweller, J. (1988). “Cognitive load during problem solving: Effects on learning.” *Cognitive Science*, 12(2), 257–285.[↵](#)
2. Cognition and Technology Group at Vanderbilt (1990). “Anchored Instruction and Its Relationship to Situated Cognition.” *Educational Researcher*, 19(6), 2–10. See also: Lave, J., & Wenger, E. (1991). *Situated Learning: Legitimate Peripheral Participation*. Cambridge University Press.[↵](#)
3. Ginting, D., Woods, R. M., Barella, Y., Limanta, L. S., Madkur, A., & How, H. E. (2024). “The Effects of Digital Storytelling on the Retention and Transferability of Student Knowledge.” *SAGE Open*. (Study population: n=41 pre-service teachers, early twenties, Indonesian university — general learning-science evidence for narrative framing, not programming-specific or age-matched to this activity’s audience.)[↵](#)
4. Sweller, J., & Cooper, G. A. (1985). “The Use of Worked Examples as a Substitute for Problem Solving in Learning Algebra.” *Cognition and Instruction*, 2(1), 59–89. (Original domain is algebra; extrapolated here to programming instruction.)[↵](#)
5. Hulleman, C. S., & Harackiewicz, J. M. (2009). “Promoting Interest and Performance in High School Science Classes.” *Science*, 326(5958), 1410–1412. (Utility-value intervention: connecting material to its personal or future relevance measurably improved both interest and grades, not self-reported enjoyment alone.)[↵](#)

6. IBM, "What is a qubit?" [ibm.com/think](https://www.ibm.com/think/topics/qubit), Schneider, J. & Smalley, I. (2024, updated 2026). <https://www.ibm.com/think/topics/qubit> — see also NIST, "What Is Superposition?" <https://www.nist.gov/video/what-superposition>[↵](#)
7. IBM Quantum Learning, "Grover's algorithm" module. <https://quantum.cloud.ibm.com/learning/en/modules/computer-science/grovers>[↵](#)
8. Pavlus, J. "Why Is Quantum Computing So Hard to Explain?" *Quanta Magazine*, June 8, 2021 (quoting Scott Aaronson on the "central fallacy" of popular quantum-computing explanations). <https://www.quantamagazine.org/why-is-quantum-computing-so-hard-to-explain-20210608/>[↵](#)
9. Grover, L. K. (1996). "A fast quantum mechanical algorithm for database search." *Proceedings of the 28th Annual ACM Symposium on Theory of Computing* (STOC '96), 212–219. Preprint: <https://arxiv.org/abs/quant-ph/9605043> — optimality of the $\sqrt{N}$ bound established in Bennett, C. H., Bernstein, E., Brassard, G., & Vazirani, U. (1997), "Strengths and Weaknesses of Quantum Computing," *SIAM Journal on Computing*, 26(5).[↵](#)
10. Rehman, M. U. "Top Global Banks Exploring Quantum Technologies in 2026." *The Quantum Insider*, March 27, 2026. <https://thequantuminsider.com/2026/03/27/15-plus-global-banks-probing-the-wonderful-world-of-quantum-technologies/> — HSBC (Group Head of Quantum Technologies; IBM/Quantinuum trials), Barclays (quantum research since 2017, Chief Technology Office), Goldman Sachs (dedicated quantum research team), and JPMorgan Chase (certified quantum randomness, QAOA demonstrations, a quantum-secured network between data centres) all independently verified in this source as of publication.[↵](#)
11. Rehman, M. U. "Top Global Banks Exploring Quantum Technologies in 2026." *The Quantum Insider*, March 27, 2026. <https://thequantuminsider.com/2026/03/27/15-plus-global-banks-probing-the-wonderful-world-of-quantum-technologies/> — HSBC (Group Head of Quantum Technologies; IBM/Quantinuum trials), Barclays (quantum research since 2017, Chief Technology Office), Goldman Sachs (dedicated quantum research team), and JPMorgan Chase (certified quantum randomness, QAOA demonstrations, a quantum-secured network between data centres) all independently verified in this source as of publication.[↵](#)
12. IBM Quantum Learning, "Grover's algorithm" module. <https://quantum.cloud.ibm.com/learning/en/modules/computer-science/grovers>[↵](#)
13. Grover, L. K. (1996). "A fast quantum mechanical algorithm for database search." *Proceedings of the 28th Annual ACM Symposium on Theory of Computing* (STOC '96), 212–219. Preprint: <https://arxiv.org/abs/quant-ph/9605043> — optimality of the $\sqrt{N}$ bound established in Bennett, C. H., Bernstein, E., Brassard, G., & Vazirani, U. (1997), "Strengths and Weaknesses of Quantum Computing," *SIAM Journal on Computing*, 26(5).[↵](#)
14. IBM Quantum Learning, "Grover's algorithm" module. <https://quantum.cloud.ibm.com/learning/en/modules/computer-science/grovers>[↵](#)

15. Shor, P. W. (1994). "Algorithms for quantum computation: discrete logarithms and factoring." *Proceedings 35th Annual Symposium on Foundations of Computer Science*, 124–134. See also Shor, P. W. (1997), "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer," *SIAM Journal on Computing*, 26(5), 1484–1509 (the journal version). Cited here only for the Shor's-algorithm flash-forward (§6); the build itself never runs this algorithm.↵
16. NIST. "NIST Releases First 3 Finalized Post-Quantum Encryption Standards." Aug 13, 2024. <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards> (FIPS 203 / ML-KEM, FIPS 204 / ML-DSA, FIPS 205 / SLH-DSA.)↵
17. Federal Reserve. "'Harvest Now, Decrypt Later': Examining Post-Quantum [Cryptography Migration]." FEDS Working Paper, 2025. <https://www.federalreserve.gov/econres/feds/files/2025093pap.pdf>↵
18. NIST. "NIST Releases First 3 Finalized Post-Quantum Encryption Standards." Aug 13, 2024. <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards> (FIPS 203 / ML-KEM, FIPS 204 / ML-DSA, FIPS 205 / SLH-DSA.)↵
19. NSA. "Commercial National Security Algorithm Suite 2.0" FAQ. https://media.defense.gov/2022/Sep/07/2003071836/-1/-1/0/CSI_CNSA_2.0_FAQ_.PDF — applies to US National Security Systems, cited here as evidence of migration urgency, not as a UK compliance requirement.↵
20. Cuthbert, M. (Director, National Quantum Computing Centre). "The UK has thrown down the gauntlet on quantum." NQCC blog, March 20, 2026. <https://www.nqcc.ac.uk/blogs/the-uk-has-thrown-down-the-gauntlet-on-quantum/>↵
21. UK Quantum Skills Taskforce Report. gov.uk, first published May 13, 2025 (updated June 24, 2025). <https://www.gov.uk/government/publications/uk-quantum-skills-taskforce-report/uk-quantum-skills-taskforce-report>↵
22. Schmidt, R. A., & Bjork, R. A. (1992). "New Conceptualizations of Practice: Common Principles in Three Paradigms Suggest New Concepts for Training." *Psychological Science*, 3(4), 207–217. (Varied, learner-directed practice conditions — as opposed to fixed, repeated drills — produce better retention and transfer, even though they often feel harder and less fluent during the practice itself.)↵